

Unit 1 · Topic Notes

Architecture Patterns — Monolith vs. Microservices, API-First Design

Course: Full Stack Application Development with MERN

Learning Objectives

1. Explain what a monolithic architecture is and why it's the right choice for this course
2. Explain microservices at a conceptual level and when real teams use them
3. Apply "API-first" thinking: design the contract before writing backend code
4. Draw their own MERN capstone's architecture diagram

1. Set Expectations First

You will build a monolith this semester. That is not a lesser choice — it is the correct engineering decision for a small team on a 4-month timeline. Microservices exist to solve problems you don't have yet.

2. Monolithic Architecture

A monolith is a single deployable unit: one Express server, one codebase, one deployment, serving the API (and often the built React app too).

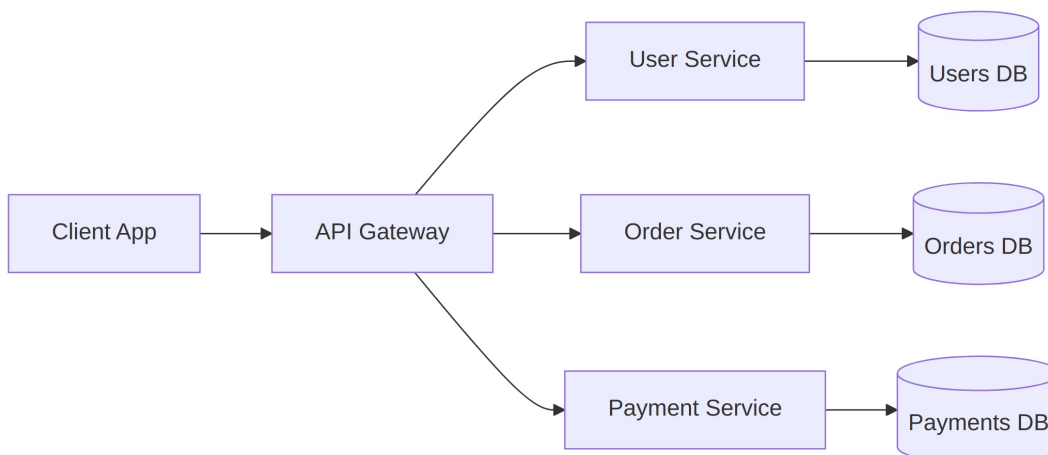


Monolith architecture diagram

Pros: simple to develop, simple to deploy (one Render/Vercel service), easy to debug, no network calls between internal parts, perfect for small teams. **Cons:** everything scales together, one bug can crash the whole app, harder to split work cleanly across a very large team.

3. Microservices Architecture

Each business capability (Users, Orders, Payments, Notifications) is its own small service with its own database, deployed and scaled independently.



Microservices architecture diagram

Pros: independent scaling, independent deployments, teams can work in parallel without stepping on each other,

one service crashing doesn't necessarily kill the rest. **Cons:** massive operational complexity (service discovery, inter-service networking, distributed data consistency), overkill below a certain team/traffic size, much harder to debug (a single user action might touch 4 services).

Companies like Netflix and Amazon run microservices because they have hundreds of engineers and need independent teams. A 4-person capstone team with one database is solving a problem that doesn't exist yet — and creating five new ones (auth between services, networking, five separate deployments) that do.

4. Decision Table

Factor	Favor Monolith	Favor Microservices
Team size	1–10 devs	50+ devs across teams
Codebase complexity	Simple-medium domain	Many independent business domains
Deployment maturity	Learning CI/CD	Mature DevOps/Kubernetes practice
Traffic	Low-medium	Massive, uneven load per feature
This course	Yes	—

5. API-First Design

Instead of writing backend code first and “figuring out the frontend later,” define the **contract** — the exact endpoints, request/response shapes — before either side is built. Frontend and backend teammates can then work in parallel against an agreed contract.

5.1 Example API Contract (write this before coding)

Method	Endpoint	Request Body	Response
GET	/api/products	—	<code>[{ "_id", "name", "price", "image" }]</code>
POST	/api/cart	<code>{ "productId", "quantity" }</code>	<code>{ "cartId", "items": [...], "total" }</code>
POST	/api/orders	<code>{ "cartId", "address" }</code>	<code>{ "orderId", "status": "pending" }</code>

5.2 Minimal OpenAPI-Style Snippet (Optional)

```
paths:
  /api/products:
    get:
      summary: List all products
      responses:
        '200':
          description: Array of product objects
```

Even without full Swagger tooling, writing this table on Day 1 prevents the classic “the frontend expects `productName` but the backend sends `name`” bug that eats hours in week 6.

6. Hands-On Lab (30 min, individual or pairs)

Using your own capstone idea (or the campus lost & found app from earlier topics):

1. Draw your system architecture as a monolith diagram (Mermaid or paper) — Browser → Express → Database
2. Write an API contract table for your 3 most important endpoints (method, path, request body, response shape)
3. In 2–3 sentences, justify *why* a monolith is the right call for your project this semester

7. Quick Assessment

1. Name one advantage and one disadvantage of microservices.
 2. Why is a monolith usually the right choice for a 4-month, 4-person project?
 3. What problem does “API-first design” specifically prevent?
 4. In the monolith diagram, what are the three main boxes?
 5. What does an API gateway do in a microservices setup?
-

Notes prepared for Full Stack Learning — MERN course, Unit 1.